

PDF2MD Major Refactor Plan

Research baseline: 28 August 2026

Architecture, security, performance, and documentation

Executive decision

PDF2MD should become a modular, content-aware document pipeline with an explicit backend router:

1. **Text-based PDF:** prefer PDFoxide as the primary Rust-native extractor for text, chars/words/lines, coordinates, tables, images, and Markdown-oriented ingestion. Use PDFium for visual verification and as a bounded fallback when PDFoxide reports unsupported or low-confidence extraction.
2. **Image-only or insufficient-text PDF:** render with PDFium and use **faster-paddle** for OCR.
3. **Every page:** run the mandatory PP-DocLayout layout stage. The selected layout model must be configurable, pinned, and recorded in the manifest.
4. **Reconstruction:** consume a backend-neutral page IR. LLM reconstruction is optional and must never be the authority for legal tokens.
5. **Reporting:** produce Markdown plus a concise XeLaTeX/PDF quality report, with provenance, routing decisions, quality gates, and unresolved review items.

This is a plan, not a claim that the refactor is already implemented.

Important constraint: Ponytail skill

The requested `ponytail` skill is not installed in the active Hermes profile; `skill_view(name="ponytail")` returned an unavailable-skill result. No steps below are attributed to that skill. The plan instead uses the verified repository state, official PDFium/PaddleOCR documentation, the web-research workflow, and PDF report workflow. Install or provide the skill before implementation if it contains project-specific rules that must override this plan.

Evidence baseline

PDFoxide and PDFium

The supplied Rust research report recommends PDFoxide as the preferred extractor for text-based PDFs, subject to independent corpus validation. Its reported API surface includes text, chars, words, lines, tables, images, and Markdown-oriented extraction. Upstream benchmark claims are treated as hypotheses, not acceptance criteria. PDFoxide therefore becomes the default native-text backend in this plan.

PDFium remains the visual fidelity and recovery backend. The upstream PDFium README identifies PDFium as the PDF library used by Chromium. It uses Chromium's GN/Ninja build system and exposes stable embedder headers under `public/`; code outside that directory may change without compatibility promises. The standalone `pdfium_test` can parse and rasterize PDFs, while PDFium's test suite includes unit, embedder, corpus, JavaScript, and pixel tests. The upstream README also recommends pixel tests for rendering changes and asks test cases to be minimal and free of copyright issues.[1]

For this repository, the practical integration should be through the maintained Rust `pdfium-render` wrapper rather than embedding the full Chromium build inside PDF2MD. Its documentation exposes page rendering,

text/image extraction, document introspection, and runtime binding. It supports a packaged dynamic library, system library fallback, or static linking; the library itself is not bundled by the wrapper.[2]

`pypdfium2` is a useful reference for behavior and packaging, not the preferred runtime here. It describes PDFium as a library for rendering, inspection, manipulation, and creation, exposes both helpers and the raw API, and warns that bindings and the out-of-tree PDFium binary must remain ABI-compatible.[3]

PP-DocLayout and PP-StructureV3

PP-StructureV3 combines layout analysis, OCR, preprocessing, table recognition, seal recognition, formula recognition, and optional chart parsing. The official documentation describes multi-column reading-order recovery and Markdown output. The layout detector includes categories such as title, paragraph, text, page number, header, footer, formula, image, table, seal, chart, and sidebar content.[4]

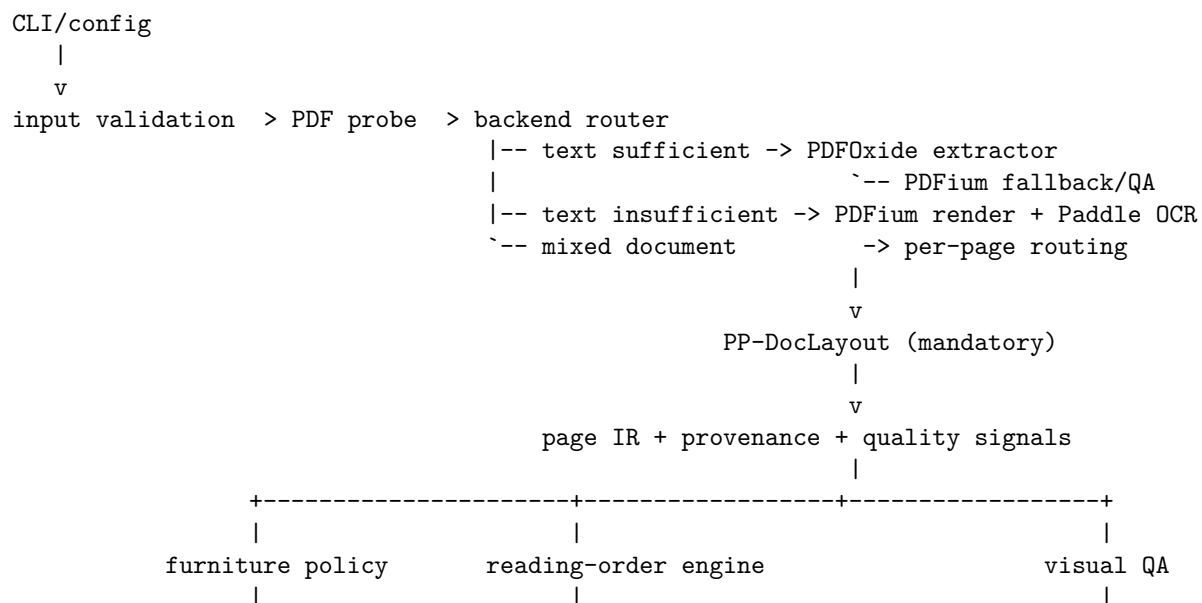
PP-DocLayout is therefore a **mandatory shared stage**, not an OCR fallback. It must classify regions before furniture removal, reading-order reconstruction, and visual review. The official page reports separate model trade-offs and explicitly notes that its benchmark numbers are measured on a self-built document-layout set; those numbers must not be treated as PDF2MD acceptance thresholds.[4]

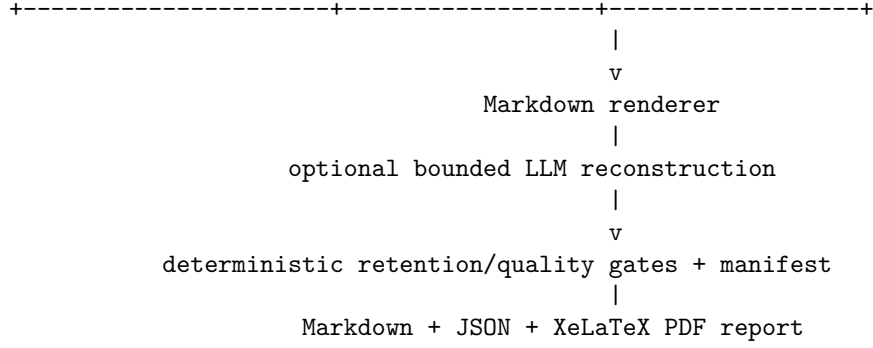
Current repository evidence

The current Rust binary has a single `main.rs` that orchestrates OCR, furniture annotation, reconstruction, and reporting. OCR currently shells out through `scripts/ocr_helper.py`, writes page JSON, and uses a sequential page loop with per-process temporary storage. Reconstruction consumes page JSON and an LLM API. The repository already contains protected-token validation, deterministic fallback, quality metrics, and a regression manifest, but the current configuration still uses generic `api.example.com/model` placeholders and the public README documents an OCR-first architecture.

The tracked repository scan found no committed `.env`, private key, home-directory path, API token, email address, Telegram ID, or user IP. It did find the POJK fixture name and sample legal tokens, which are domain-specific rather than personal data. `data/frequency_dict.txt` contains ordinary lexical entries such as “telegram” and “password”; these are dictionary words, not credentials. A history and binary/artifact scan must still be run before the refactor is merged.

Target architecture





Modules

Module	Responsibility	Boundary
probe	Detect page count, metadata, text-layer coverage, encrypted/unsupported PDFs	No OCR, no LLM
pdfoxide_backend	Primary native text, chars/words/lines, tables, images, Markdown-oriented extraction	Produces backend-neutral observations
pdfium_backend	Page render, visual QA, complex-PDF fallback, PDFium version	Never silently replaces failed extraction
ocr_backend	faster-paddle execution and fallback/retry policy	Never silently returns a blank page
layout_backend	Mandatory PP-DocLayout inference	Emits typed regions and model provenance
routing	Document- or page-level backend decision	Deterministic, explainable decision record
ir	Stable page/document intermediate representation	Versioned schema; no provider-specific fields in core structures
furniture	Header/footer/page-marker classification	Preserves ambiguous content
reading_order	Region ordering, columns, lists, tables	Emits confidence and review flags
render_md	Deterministic Markdown from IR	No token invention
reconstruct	Optional LLM formatting/repair	Protected-token contract and bounded fallback
quality	Coverage, token recall, CER/WER when gold exists, visual flags	Hard gates are explicit
report	Markdown and XeLaTeX report generation	Reproducible, no secrets
artifacts	Cache, manifests, page images, logs	Retention and redaction policy

Routing contract

Text-layer detection

Probe each page with PDFium. A page is “text sufficient” only when it has a non-trivial number of visible text objects, non-empty extracted text, usable coordinates, and no contradiction from raster/content signals. Do not use a single character-count threshold as a universal truth. Record the evidence: `text_chars`, `text_objects`, `visible_glyph_ratio`, `coverage_estimate`, and `probe_version`.

Recommended first implementation:

- **Document probe:** inspect all pages cheaply through PDFium.
- **Page routing:** route independently to support mixed PDFs.
- **Native path:** extract spans and images with PDFium, render only for visual QA or when layout confidence is low.
- **OCR path:** render the page with PDFium at a configured DPI, run **faster-paddle**, then pass image plus OCR boxes to PP-DocLayout.
- **Ambiguous path:** run both native extraction and OCR on a bounded sample or the affected page, compare normalized token sets, and mark review if they disagree materially.

Mandatory PP-DocLayout policy

PP-DocLayout must run after page acquisition and before furniture filtering. The model name, model package digest/version, inference engine, device, and timing must be in each run manifest. Do not hard-code a benchmark score as a pass gate. Use a configured default plus an explicit `--layout-model` override.

The first production profile should prefer a balanced model for throughput and make the high-precision model available for legal-review mode. The choice must be validated on the project’s own corpus; official model benchmarks are comparative signals, not acceptance evidence.

Performance refactor

Hot-path changes

1. Replace the monolithic `main.rs` orchestration with library modules and thin CLI commands.
2. Probe the PDF once and reuse the page manifest; do not repeatedly call `pdfinfo`, re-render, or rescan the same page.
3. Use bounded worker pools for independent pages. Keep PDFium document access isolated behind a thread-safe policy; do not assume a single PDFium document handle is safe across threads without testing.
4. Batch **faster-paddle** calls where the helper supports it; otherwise use a long-lived worker process rather than spawning Python for every page.
5. Cache page renderings and backend results by source hash, page number, DPI, backend version, model identity, and configuration hash.
6. Stream page results into a manifest instead of retaining the whole document in memory. Keep deterministic page ordering at the output boundary.
7. Separate CPU-bound extraction/OCR workers from network-bound reconstruction workers, each with its own concurrency limit.
8. Make retries page-scoped and exponential; never rerun successful pages after a later page fails.
9. Preserve raw page evidence and derived IR separately, enabling reconstruction reruns without repeating OCR or PDFium extraction.
10. Add timing for probe, PDFium extraction, render, layout, OCR, reconstruction, validation, and report compilation.

Proposed CLI

```
pdf2md inspect      --input INPUT.pdf --out RUN/
pdf2md acquire      --input INPUT.pdf --run RUN/
pdf2md layout       --run RUN/ --model PP-DocLayout-M
pdf2md normalize    --run RUN/
pdf2md render-md    --run RUN/ --out OUTPUT/
pdf2md reconstruct  --run RUN/ --out OUTPUT/
pdf2md validate     --run RUN/
pdf2md report       --run RUN/ --out REPORT.pdf
pdf2md run          --input INPUT.pdf --out OUTPUT/ --report REPORT.pdf
```

`run` is a convenience composition; each stage remains independently rerunnable for performance, debugging, and testability.

Repository sanitization

Before implementation changes, run a security and portability gate:

- Search tracked files and Git history for secrets, private keys, bearer tokens, personal paths, email addresses, chat IDs, IP addresses, local model endpoints, and user-specific filenames.
- Replace private examples with `examples/` fixtures using synthetic text and openly redistributable assets, or generate fixtures at test time.
- Remove personal source PDFs and generated OCR/Markdown/PDF artifacts from the repository and history where they are committed. If removal requires rewriting public history, obtain explicit confirmation first and coordinate a force-push.
- Keep only `.env.example` with placeholder variable names; never include working credentials or real provider URLs that imply a user's private account.
- Use neutral names such as `input.pdf`, `sample-contract.pdf`, and `output/run-001/`; do not encode a person's name, employer, address, or private document title in tests, comments, fixtures, or README commands.
- Add automated secret scanning in CI and a repository portability test that rejects `/home/`, `/tmp/`, `.hermes`, private IPs, credential markers, and known user identifiers in tracked text.
- Review Git LFS/binary objects and tags, not only the current checkout.

Quality and release gates

Hard gates

- Every requested page has a result and a manifest entry.
- No page is silently blank; sparse pages are retried or fail visibly.
- PP-DocLayout ran for every page, or the manifest contains an explicit, reviewable exception.
- Protected-token recall is 100% for the configured legal-token inventory.
- Markdown validation passes and output page order is deterministic.
- No unresolved credential/private-data scan finding.
- Report compilation succeeds and contains the run ID, versions, decisions, and failure list.

Measured gates

When ground truth exists, record CER, WER, token precision/recall, omission rate, reading-order errors, table-region accuracy, and visual comparison status. Keep thresholds corpus-specific and versioned. Do not claim CER/WER from a document without a ground-truth transcription.

Migration phases

Phase	Deliverable	Exit condition
0. Baseline	Freeze current tests, manifests, and benchmark outputs	Reproducible baseline report
1. Sanitize	Remove personal/private examples; add scans and synthetic fixtures	Clean tree/history decision recorded
2. IR	Define versioned document/page/region schema	Native and OCR adapters compile to same IR
3. PDFium	Add probe, native extraction, rendering, dynamic-library diagnostics	Text PDF fixture bypasses OCR correctly
4. Layout	Integrate mandatory PP-DocLayout and model manifest	Every page has regions and timing
5. Router	Add per-page text sufficiency and mixed-document routing	Routing tests cover text, scan, sparse, mixed
6. Modular workers	Split stages, bounded pools, cache keys, retries	Performance benchmark improves without quality loss
7. Quality	Add gates, golden fixtures, visual/pixel checks, secret scan	CI fails on silent loss or private data
8. Reporting	Add XeLaTeX report generation and artifact retention	PDF report compiles and is content-verified
9. README	Rewrite concise README after major refactor	Commands describe universal architecture only

Risks and decisions

- **Rust-first policy:** PDFioxide is the default native-text engine and the core pipeline remains Rust. PDFium is an optional, isolated native component used for rendering, visual QA, and bounded recovery; its absence must not break pure-Rust text extraction. Diagnose its dynamic-library/ABI availability explicitly and do not hide a missing library behind an OCR fallback.
- **Text-layer quality:** a PDF can contain a bad or partial text layer. Routing must be page-aware and compare native evidence with layout/OCR signals.
- **Layout cost:** PP-DocLayout is mandatory, so benchmark CPU/GPU profiles and expose model selection rather than pretending the cost is free.
- **Legal fidelity:** LLM output remains a formatting aid. Deterministic source evidence, protected-token checks, and visible review flags outrank fluency.
- **Copyright/privacy:** fixtures must be synthetic or legally redistributable; official PDFium guidance also recommends minimal pixel-test cases without copyright issues.[1]

Sources

[1] PDFium upstream README: [https://pdfium.googlesource.com/pdfium/+main/README.md](https://pdfium.googlesource.com/pdfium/+/main/README.md)

[2] `pdfium-render` Rust wrapper: <https://github.com/ajrcarey/pdfium-render>

[3] `pypdfium2` bindings and packaging/ABI guidance: <https://github.com/pypdfium2-team/pypdfium2>

[4] PaddleOCR PP-StructureV3 / PP-DocLayout documentation: <https://paddlepaddle.github.io/PaddleOCR/latest/en/vers/StructureV3.html>

[5] PaddleOCR project overview: <https://github.com/PaddlePaddle/PaddleOCR>